

# **C++ - CONSTRUCTOR & DESTRUCTOR**

Dr. S. Vinila Jinny



# Constructor

- Constructors are special class members which are called by the compiler every time an object of that class is instantiated. Constructors have the same name as the class and may be defined inside or outside the class definition. There are 3 types of constructors:
  - Default constructors
  - Parameterized constructors
  - Copy constructors



# Default Constructor

- A constructor which has no argument is known as default constructor. It is invoked at the time of creating object.

```
#include <iostream>
class Employee
{
    public:
        Employee()
        {
            cout<<"Default Constructor
Invoked"<<endl;
        }
};
int main(void)
{
    Employee e1; //creating an object of
Employee
    Employee e2;
    return 0;
}
```

Output:

Default Constructor Invoked  
Default Constructor Invoked



# Parametrized Constructor

- A constructor which has parameters is called parameterized constructor.
- It is used to provide different values to distinct objects.

```
#include <iostream>
class Employee {
public:
    int id;
    string name;
    float salary;
    Employee(int i, string n, float s)
    {
        id = i;
        name = n;
        salary = s;
    }
    void display()
    {
        cout<<id<<" "<<name<<" "<<salary<<endl;
    }
};
```

```
int main(void) {
    Employee e1 =Employee(101, "Sonoo", 890000);
    Employee e2=Employee(102, "Nakul", 59000);
    e1.display();
    e2.display();
    return 0;
}
```

Output:

```
101 Sonoo 890000
102 Nakul 59000
```



# Copy Constructor

- Copy constructor is a type of constructor which is used to create a copy of an already existing object of a class type.
- A copy constructor is invoked when an existing object is passed as a parameter.



```

#include<iostream>

using namespace std;

class Employee
{
private:

int salary, experience;

public:

Employee(int x1, int y1)
{
salary = x1;
experience = y1;
}

Employee(const Employee &new_employee)
{
salary = new_employee.salary;
experience = new_employee.experience;
}

```

# Example

```

void display()
{
cout << "Salary: " << salary << endl;
cout << "Years of experience: " << experience << endl;
}
};

int main()
{
Employee employee1(34000, 2);
Employee employee2 = employee1;
cout << "Employee1 using parameterized constructor : \n";
employee1.display();
cout << "Employee2 using copy constructor : \n";
employee2.display();
return 0;
}

```

Output:

Employee1 using parameterized constructor :

Salary: 34000

Years of experience: 2

Employee2 using copy constructor :

Salary: 34000

Years of experience: 2





# Characteristics of Constructor

- They should be declared in the public section
- They do not have any return type, not even void
- They get automatically invoked when the objects are created
- They cannot be inherited though derived class can call the base class constructor
- Like other functions, they can have default arguments
- You cannot refer to their address
- Constructors cannot be virtual





# Destructor

- Destructor is a member function that is instantaneously called whenever an object is destroyed.
- The destructor is called automatically by the compiler when the object goes out of scope i.e. when a function ends the local objects created within it also gets destroyed with it.
- The destructor has the same name as the class name, but the name is preceded by a tilde(~).
- A destructor has no return type and receives no parameters.



# Destructor

- Syntax:

~constructor-name();

```
class String {  
    private:
```

```
        char* s;  
        int size;
```

```
    public:
```

```
        String(char*); // constructor  
        ~String(); // destructor
```

```
};
```

```
String::String(char* c)  
{
```

```
    size = strlen(c);  
    s = new char[size + 1];  
    strcpy(s, c);
```

```
}
```

```
String::~~String() { delete[] s; }
```



# Characteristics of a Destructor in C++

- A destructor deallocates memory occupied by the object when it's deleted.
- A destructor cannot be overloaded. In function overloading, functions are declared with the same name in the same scope, except that each function has a different number of arguments and different definitions. But in a class, there is always a single destructor and it does not accept any parameters, hence, a destructor cannot be overloaded.
- A destructor is always called in the reverse order of the constructor. In C++, variables and objects are allocated on the Stack. The Stack follows LIFO (Last-In-First-Out) pattern. So, the deallocation of memory and destruction is always carried out in the reverse order of allocation and construction. This can be seen in code below.
- A destructor can be written anywhere in the class definition. But to bring an amount of order to the code, a destructor is always defined at the end of the class definition.

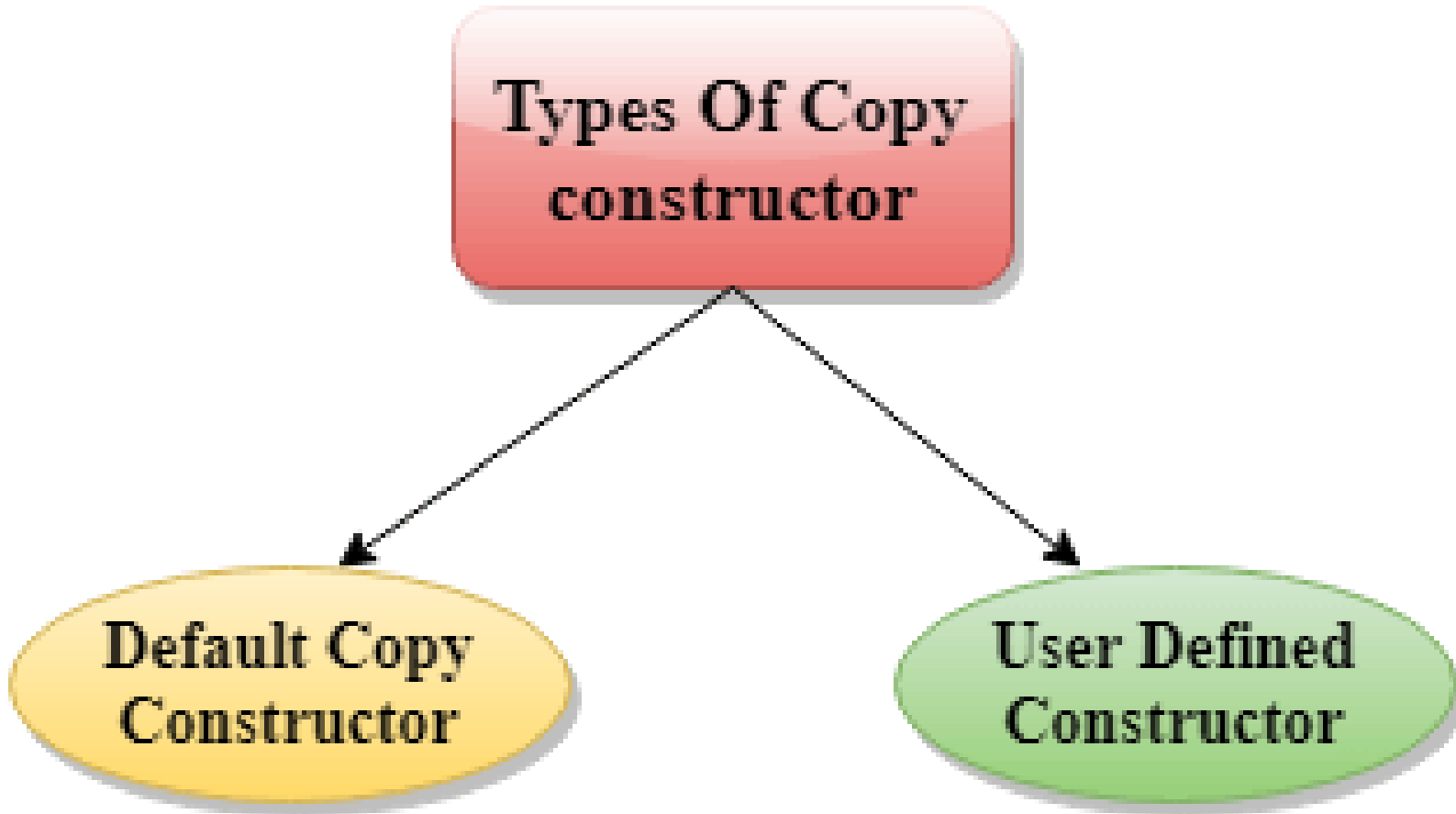


# More About Copy Constructor

- A Copy constructor is an **overloaded** constructor used to declare and initialize an object from another object.
- Copy Constructor is of two types:
- **Default Copy constructor:** The compiler defines the default copy constructor. If the user defines no copy constructor, compiler supplies its constructor.
- **User Defined constructor:** The programmer defines the user-defined constructor.



## Types Of Copy constructor



```
graph TD; A[Types Of Copy constructor] --> B(Default Copy Constructor); A --> C(User Defined Constructor);
```

Default Copy  
Constructor

User Defined  
Constructor

ONLINE  
COURSE



# Syntax & Example

Class\_name(const class\_name &old\_object);

class A

{

    A(A &x) // copy constructor.

{

    // copyconstructor.

}

}

- **A a2(a1);**

- **A a2 = a1;**

}

**a1 initialises the a2 object.**

```
#include <iostream>
using namespace std;
class A
{
    public:
        int x;
        A(int a)
        {
            x=a;
        }
        A(A &i) {
            x = i.x;
        }
};
```

## Example

```
int main()
{
    A a1(20);
    A a2(a1);
    cout<<a2.x;
    return 0;
}
```

Output:  
20



# When Copy Constructor is called

Copy Constructor is called in the following scenarios:

- When we initialize the object with another existing object of the same class type.
- For example, `Student s1 = s2`, where `Student` is the class.
- When the object of the same class type is passed by value as an argument.
- When the function returns the object of the same class type by value.





# Types of copies produced by the constructor

- Shallow copy
- Deep copy



# Shallow Copy

- The default copy constructor can only produce the shallow copy.
- A Shallow copy is defined as the process of creating the copy of an object by copying data of all the member variables as it is.



```
#include <iostream>

using namespace std;

class Demo
{
    int a;
    int b;
    int *p;
public:
    Demo()
    {
        p=new int;
    }
    void setdata(int x,int y,int z)
    {
        a=x;
        b=y;
        *p=z;
    }
};
```

# Example

```
void showdata()
{
    cout << "value of a is : " <<a<< endl;
    cout << "value of b is : " <<b<< endl;
    cout << "value of *p is : " <<*p<< endl;
}

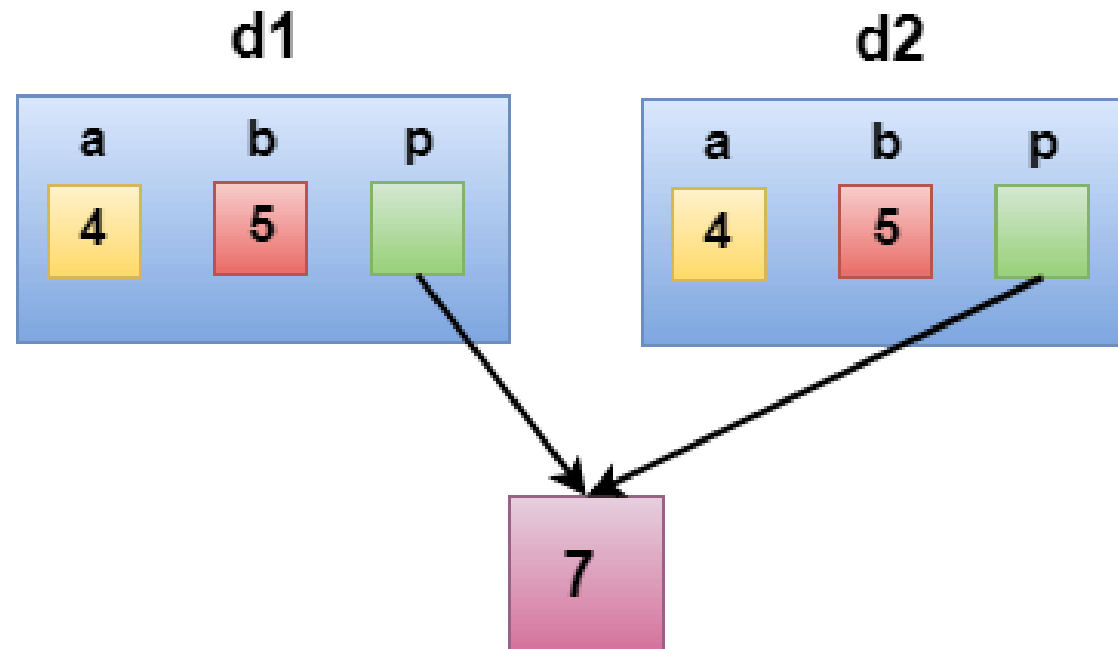
int main()
{
    Demo d1;
    d1.setdata(4,5,7);
    Demo d2 = d1;
    d2.showdata();
    return 0;
}
```

Output:

value of a is : 4  
value of b is : 5  
value of \*p is : 7



# Shallow copy



# Deep Copy

- Deep copy dynamically allocates the memory for the copy and then copies the actual value, both the source and copy have distinct memory locations.
- In this way, both the source and copy are distinct and will not share the same memory location.
- Deep copy requires us to write the user-defined constructor.



```

#include <iostream>
using namespace std;
class Demo
{
    public:
    int a;
    int b;
    int *p;

    Demo()
    {
        p=new int;
    }

    Demo(Demo &d)
    {
        a = d.a;
        b = d.b;
        p = new int;
        *p = *(d.p);
    }
}

```

```

void setdata(int x,int y,int z)
{
    a=x;
    b=y;
    *p=z;
}
void showdata()
{
    cout << "value of a is : " <<a<< endl;
    cout << "value of b is : " <<b<< endl;
    cout << "value of *p is : " <<*p<< endl;
}
};
int main()
{
    Demo d1;
    d1.setdata(4,5,7);
    Demo d2 = d1;
    d2.showdata();
    return 0;
}

```

Output:

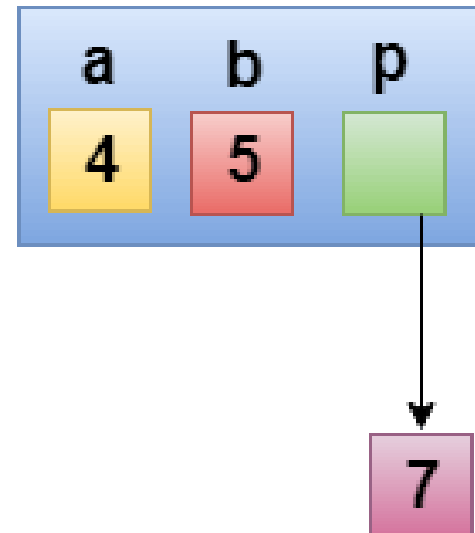
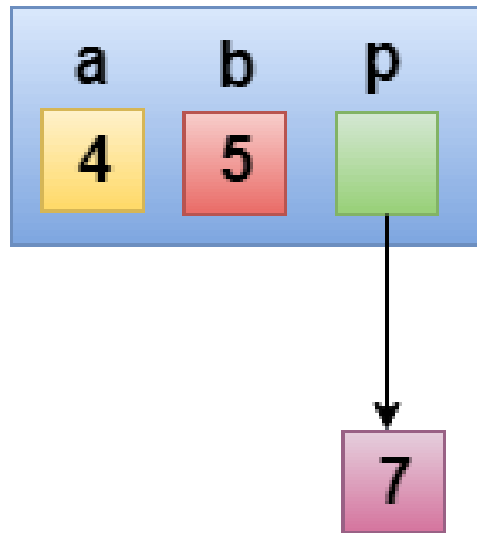
```

value of a is : 4
value of b is : 5
value of *p is : 7

```



# Deep Copy



# THANK YOU

